

QB 07: Object-Oriented Programming with Java — Question Bank with Answer Keys

Course: Object-Oriented Programming using Java (Theory + Lab)

Target: VTU MCA Semester 2 (IPCC)

Scheme: VTU 2022 / 2024 Scheme

Contents: Runtime Polymorphism, Abstract Classes vs Interfaces, Multithreading & Synchronization, Custom Exceptions, Collections Framework, 15 MCQs with Explanations

Syllabus Outline (VTU 5 Modules)

- **Module 1:** Java Architecture (JVM, JRE, JDK, JIT, Bytecode), Classes, Objects, Constructors, `this` and `super` keywords, Garbage Collection.
- **Module 2:** Inheritance (Single, Multilevel, Hierarchical), Method Overriding vs Overloading, Dynamic Method Dispatch, `final` keyword, Abstract Classes, and Interfaces.
- **Module 3:** Packages and Access Modifiers, Exception Handling (`try` , `catch` , `finally` , `throw` , `throws` , Custom Exceptions, `try-with-resources`).
- **Module 4:** Multithreading: Life Cycle of a Thread, `Thread` vs `Runnable` , Thread Synchronization, Inter-thread Communication (`wait()` , `notify()`), Deadlocks.
- **Module 5:** Java Collections Framework (`ArrayList` , `LinkedList` , `HashSet` , `HashMap`), Generics, Lambda Expressions, and Streams API.

SECTION 1: High-Yield MCQs with Answer Key

Q1. Why is Java considered platform-independent?

- A) It runs directly on CPU registers without OS intervention
- B) Java compiler generates platform-neutral intermediate Bytecode (`.class`), which is interpreted/JIT-compiled by the platform-specific JVM
- C) It is written in C++
- D) It only supports Linux

Answer: B

Explanation: The "Write Once, Run Anywhere" (WORA) capability stems from Bytecode executed by the Java Virtual Machine (JVM) implemented for each target OS.

Q2. What mechanism enables Dynamic Method Dispatch (Runtime Polymorphism) in Java?

- A) Method overloading
- B) Method overriding where an overridden method call is resolved at runtime based on the actual object referenced
- C) Static variable binding
- D) Package imports

Answer: B

Explanation: When a parent class reference variable points to a subclass object, calling an overridden method invokes the subclass version dynamically at runtime.

Q3. Can an `abstract` class in Java contain concrete (implemented) methods?

- A) No, it must contain only abstract methods
- B) Yes, an abstract class can have both abstract and non-abstract (concrete) methods
- C) Only if the class is marked `final`
- D) Only static methods are permitted

Answer: B

Explanation: Unlike traditional interfaces, abstract classes can provide common base implementations alongside abstract method contracts.

Q4. What happens if an exception is thrown in a `try` block and no matching `catch` block exists?

- A) The program ignores the error
- B) The `finally` block (if present) executes, and then unhandled exception propagation terminates the thread
- C) The JVM restarts automatically
- D) Compilation error occurs

Answer: B

Explanation: The `finally` block is guaranteed to execute before the exception propagates up the call stack to the default uncaught exception handler.

Q5. Which interface must be implemented to allow an object to be executed as a background thread?

- A) `java.lang.Callable`
- B) `java.lang.Runnable`
- C) `java.io.Serializable`
- D) `java.lang.Cloneable`

Answer: B

Explanation: Implementing `Runnable` requires defining the single `public void run()` method executed by the thread.

Q6. What is the fundamental difference between `HashMap` and `Hashtable` in Java?

- A) `HashMap` is thread-safe; `Hashtable` is not
- B) `HashMap` is non-synchronized and permits one `null` key; `Hashtable` is synchronized (legacy thread-safe) and forbids `null` keys/values
- C) `Hashtable` is faster than `HashMap`
- D) `HashMap` preserves insertion order

Answer: B

Explanation: `HashMap` is modern, unsynchronized (faster), and allows `null`. `Hashtable` is legacy, thread-safe, and rejects `null`.

Q7. What keyword prevents a class from being inherited in Java?

- A) `static`
- B) `abstract`
- C) `final`
- D) `private`

Answer: C

Explanation: Declaring a class as `final class SubClass` forbids any other class from extending it (e.g., `java.lang.String` is `final`).

Q8. What is the contract of `wait()` and `notify()` methods in Java multithreading?

- A) They can be called from any arbitrary method
- B) They must only be called from within a `synchronized` context (method or block) owning the monitor lock
- C) They belong to the `Thread` class
- D) They stop thread execution permanently

Answer: B

Explanation: `wait()`, `notify()`, and `notifyAll()` belong to `java.lang.Object` and throw `IllegalMonitorStateException` if invoked without holding the object monitor lock.

Q9. What does the `try-with-resources` statement ensure?

- A) Faster compilation
- B) Automatic closure of any resource implementing `java.lang.AutoCloseable` at block exit
- C) Suppresses all runtime errors
- D) Multithreaded execution

Answer: B

Explanation: Introduced in Java 7, `try (Resource r = new Resource())` automatically calls `r.close()` regardless of whether execution succeeds or throws an exception.

Q10. Which Java collection guarantees that elements are maintained in natural sorted order?

- A) `ArrayList`
- B) `HashSet`
- C) `TreeSet`
- D) `LinkedList`

Answer: C

Explanation: `TreeSet` (backed by a Red-Black Tree) automatically sorts elements according to their natural ordering or a supplied `Comparator`.

SECTION 2: Short Answer Concepts (4–6 Marks)

Q11. Compare Abstract Class vs Interface in Java (post-Java 8).

Answer:

Feature Abstract Class Interface (Java 8+) Inheritance A class can extend only one abstract class (Single Inheritance). A class can implement multiple interfaces. Methods Can have abstract, concrete, <code>final</code>, and <code>static</code> methods. Can have abstract methods, <code>default</code> methods, and <code>static</code> methods. Variables Can have instance variables (any access modifier: <code>private</code>, <code>protected</code>). Only <code>public static final</code> constants. Constructors Has constructors (invoked by subclass via <code>super()</code>). Cannot have constructors. Speed Slightly faster method lookup. Slightly slower dynamic interface dispatch. Design Intent "Is-A" relationship with shared state/code. "Can-Do" peripheral capability contract.	Inheritance A class can extend only one abstract class (Single Inheritance). A class can implement multiple interfaces. Methods Can have abstract, concrete, <code>final</code>, and <code>static</code> methods. Can have abstract methods, <code>default</code> methods, and <code>static</code> methods. Variables Can have instance variables (any access modifier: <code>private</code>, <code>protected</code>). Only <code>public static final</code> constants. Constructors Has constructors (invoked by subclass via <code>super()</code>). Cannot have constructors. Speed Slightly faster method lookup. Slightly slower dynamic interface dispatch. Design Intent "Is-A" relationship with shared state/code. "Can-Do" peripheral capability contract.
---	---

Q12. Explain the Life Cycle of a Thread in Java with a state transition diagram.

Answer:



Thread States (`java.lang.Thread.State`):

1. **NEW:** Thread instance created, but `start()` has not yet been called.
2. **RUNNABLE:** Thread is executing or ready in the OS scheduling queue.
3. **BLOCKED:** Waiting to acquire a monitor lock for a synchronized block/method.
4. **WAITING:** Waiting indefinitely for another thread (via `wait()` or `join()`).
5. **TIMED_WAITING:** Waiting for a specified time interval (via `sleep(ms)` or `wait(ms)`).
6. **TERMINATED:** The `run()` method has finished execution.

SECTION 3: VTU Model Practical Programs (10–12 Marks)

Q13. [VTU Model QP - Custom Exception Handling]

Write a complete Java application to simulate a Bank Account with deposit and withdrawal operations. Define a custom checked exception `InsufficientFundsException` that is thrown when a customer attempts to withdraw more than the available balance or attempts to leave a balance below the minimum required balance of ₹1,000.

Answer:

```
// 1. Custom Checked Exception
class InsufficientFundsException extends Exception {
    private double deficit;

    public InsufficientFundsException(String message, double deficit) {
        super(message);
        this.deficit = deficit;
    }

    public double getDeficit() {
        return deficit;
    }
}

// 2. Bank Account Class
class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private static final double MIN_BALANCE = 1000.00;

    public BankAccount(String accountNumber, String accountHolder, double initialDeposit) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialDeposit;
    }

    public void deposit(double amount) {
        if (amount ≤ 0) {
            System.out.println("Invalid deposit amount!");
            return;
        }
        balance += amount;
        System.out.printf("Deposited: INR %.2f | New Balance: INR %.2f\n", amount, balance);
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        System.out.printf("Attempting to withdraw: INR %.2f ... \n", amount);

        if (amount ≤ 0) {
            System.out.println("Withdrawal amount must be positive!");
            return;
        }

        // Check if withdrawal violates minimum balance requirement
        if ((balance - amount) < MIN_BALANCE) {
            double deficit = MIN_BALANCE - (balance - amount);
            throw new InsufficientFundsException(
                "Transaction Failed: Minimum balance of INR 1000 must be maintained.",
                deficit
            );
        }
    }
}
```

```
        balance -= amount;
        System.out.printf("Withdrawal Successful! Dispensed: INR %.2f | Remaining Balance: INR
    }

    public double getBalance() {
        return balance;
    }
}

// 3. Driver Main Class
public class BankAppDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("VTU2024MC01", "Kadam", 5000.00);

        System.out.println("Initial Balance: INR " + account.getBalance());
        account.deposit(2000.00); // Balance becomes 7000

        // Successful withdrawal
        try {
            account.withdraw(4000.00); // Balance becomes 3000
        } catch (InsufficientFundsException e) {
            System.err.println("Exception: " + e.getMessage());
        }

        // Unsuccessful withdrawal triggering custom exception
        try {
            account.withdraw(2500.00); // Leaves 500, which is below 1000 MIN_BALANCE!
        } catch (InsufficientFundsException e) {
            System.err.println("\n--- TRANSACTION EXCEPTION CAUGHT ---");
            System.err.println("Reason: " + e.getMessage());
            System.err.printf("Shortfall Deficit Amount: INR %.2f%n", e.getDeficit());
        } finally {
            System.out.println("\n[Finally Block] Account audit check completed.");
            System.out.printf("Final Available Balance: INR %.2f%n", account.getBalance());
        }
    }
}
```

Q14. [VTU Model QP - Multithreading & Synchronization]

Write a Java multithreaded program demonstrating thread synchronization to solve the classic Producer-Consumer problem using `wait()` and `notify()` .

Answer:

```
// Shared Buffer Resource
class SharedQueue {
    private int data;
    private boolean hasData = false;

    // Synchronized method for Producer
    public synchronized void produce(int value) {
        while (hasData) {
            try {
                wait(); // Wait until consumer consumes the existing item
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        this.data = value;
        this.hasData = true;
        System.out.println("Producer produced item: " + value);
        notify(); // Notify the waiting consumer thread
    }

    // Synchronized method for Consumer
    public synchronized int consume() {
        while (!hasData) {
            try {
                wait(); // Wait until producer produces an item
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        this.hasData = false;
        System.out.println("Consumer consumed item: " + data);
        notify(); // Notify the waiting producer thread
        return data;
    }
}

// Producer Worker Thread
class Producer implements Runnable {
    private final SharedQueue queue;

    public Producer(SharedQueue queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        for (int i = 1; i ≤ 5; i++) {
            queue.produce(i);
            try {
                Thread.sleep(400); // Simulate production time
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```



```

    }
}

// Consumer Worker Thread
class Consumer implements Runnable {
    private final SharedQueue queue;

    public Consumer(SharedQueue queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        for (int i = 1; i ≤ 5; i++) {
            queue.consume();
            try {
                Thread.sleep(800); // Simulate consumption time
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        SharedQueue queue = new SharedQueue();

        Thread producerThread = new Thread(new Producer(queue), "Producer-Thread");
        Thread consumerThread = new Thread(new Consumer(queue), "Consumer-Thread");

        producerThread.start();
        consumerThread.start();
    }
}

```

💡 Key Takeaway Checklist for Exam Day

- [] Understand difference between `throw` (explicitly throw an exception object) and `throws` (method signature declaration).
- [] For multithreading questions, always explain the monitor lock and why `wait()` / `notify()` must be inside a `synchronized` block.
- [] Remember that in Java, multiple inheritance of classes is NOT supported to avoid the Diamond Problem, but multiple inheritance of type via interfaces IS supported.